

Operation	Target Code / Function	Best Case	Average Case	Worst Case	Space Complexity	Mathematical Justification
Unamortized Insert	<code>add_item()</code>	$O(1)$	$O(1)$	$O(n)$	$O(n)$	Inserting at the end takes $O(1)$ when capacity remains. The worst case is $O(n)$ when the array reaches capacity because a new array is allocated and n existing elements are copied.
Linear Deletion	<code>remove_item_at()</code>	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Deleting the last element takes $O(1)$. Deleting at index 0 requires shifting $n - 1$ elements. The redundant inner loop adds overhead but does not change the asymptotic complexity.
Linear Search	<code>findItem()</code>	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Best case occurs when the target is at index 0. Worst case occurs when the target is at index $n - 1$ or is not present, requiring a scan of all n elements.
Array Display	<code>printAll()</code>	$O(n)$	$O(n)$	$O(n)$	$O(1)$	The function sequentially visits and displays every element from index 0 through $n - 1$, requiring n operations.
Matrix Transpose	<code>processMatrix()</code>	$O(R \times C)$	$O(R \times C)$	$O(R \times C)$	$O(R \times C)$	The nested loops visit every cell in an $R \times C$ matrix exactly once. Therefore, the number of operations is approximately $R \times C$.

						The temporary transpose matrix also requires $O(R \times C)$ space.
--	--	--	--	--	--	---